

TP pull requests : préparation du dépôt

Ce document liste le projet de base pour la séance. Compter environ 30 minutes. Les étapes 2 à 5 doivent être faites dans l'ordre : les issues doivent être créées avant toute pull request, pour que leurs numéros (#1 à #13) correspondent au catalogue des features.

1. Liste des prérequis

1. un compte GitHub, dont on récupère l'identifiant pour l'invitation ;
2. Git installé (Git for Windows sous Windows, qui fournit Git Bash et Git Credential Manager) ;
3. Node.js 22.13 ou supérieur, version LTS recommandée ;
4. un éditeur de code, VS Code par exemple.

2. Créer le dépôt

On décompresse l'archive `memo-git.zip`, puis :

```
cd memo-git
git init -b main
git add .
git commit -m "chore: initialiser le projet Mémo Git"
```

Méthode de création de dépôt avec CLI

```
gh repo create memo-git --public --source=. --remote=origin --push
```

Méthode de création Sans GitHub CLI

On crée sur GitHub un dépôt public vide nommé `memo-git` (sans README, sans `.gitignore`, sans licence), puis :

```
git remote add origin https://github.com/<proprietaire>/memo-git.git
git push -u origin main
```

Le dépôt doit être public : sur un compte GitHub gratuit, les rulesets ne sont disponibles que sur les dépôts publics, et l'offre gratuite de SonarQube Cloud analyse sans limite de taille les projets publics.

On vérifie le projet depuis un clone neuf :

```
npm ci
npm run check
npm start
```

3. Régler les fusions

Dans le dépôt : **Settings**, puis **General**, section **Pull Requests** :

1. décocher **Allow merge commits** ;
2. garder **Allow squash merging** coché, et choisir **Pull request title** dans le menu du message par défaut ;
3. décocher **Allow rebase merging** ;
4. cocher **Automatically delete head branches**.

4. Protéger la branche main

Settings, puis **Rules**, puis **Rulesets**, puis **New ruleset**, puis **New branch ruleset** :

1. **Ruleset Name** : `protection-main` ;
2. **Enforcement status** : **Active** ;
3. **Bypass list** : ajouter le rôle **Repository admin**, pour pouvoir intervenir en cas de blocage ;
4. **Target branches** : **Add target**, puis **Include default branch** ;
5. règles cochées : **Restrict deletions**, **Require a pull request before merging** avec **Required approvals** à 1 et **Require conversation resolution before merging**, **Block force pushes** ;
6. enregistrer avec **Create**.

On ne peut alors plus pousser directement sur `main`. Les deux approbations demandées en partie 2 relèvent de la consigne, la règle n'en impose qu'une.

5. Créer les issues

Le script `creer_issues.sh` crée les labels « niveau 1 », « niveau 2 », « niveau 3 », puis les issues #1 à #12 (une par feature, avec ses critères d'acceptation) et l'issue #13 commune de la partie 1. Il s'arrête sans rien créer si le dépôt contient déjà une issue ou une pull request.

```
gh auth status
./creer_issues.sh <proprietaire>/memo-git
```

Sous Windows, on le lance depuis Git Bash.

6. Connecter SonarQube Cloud

1. Sur <https://sonarcloud.io>, se connecter avec **GitHub**.
2. Importer une organisation GitHub : pour ce TP, c'est le compte qui possède le dépôt.
3. GitHub demande d'installer l'application SonarQube Cloud : choisir **Only select repositories** et sélectionner `memo-git`.
4. De retour sur SonarQube Cloud, valider la clé d'organisation proposée et choisir l'offre **Free**.
5. Sélectionner le dépôt `memo-git` et lancer la configuration ; pour la définition du nouveau code, choisir **Previous version**.
6. Attendre la fin de la première analyse automatique de `main`, puis vérifier dans **Administration**,

Analysis Method, que **Automatic Analysis** est activée.

On note l'adresse de la page du projet, à communiquer aux élèves au lancement.

Facultatif, après les premières pull requests de la partie 1 : ajouter au ruleset la règle **Require status checks to pass** avec le check `SonarCloud Code Analysis`, pour que GitHub bloque aussi la fusion quand la quality gate échoue. Le check n'apparaît dans la liste qu'une fois exécuté au moins une fois sur le dépôt.

7. Inviter les autres élèves

Settings, puis **Collaborators**, puis **Add people**, pour chacun des cinq comptes. Sur un dépôt de compte personnel, un collaborateur peut pousser des branches et fusionner des pull requests, mais pas modifier les réglages. Une invitation non acceptée expire au bout de 7 jours

8. Attribuer les numéros E1 à E5

Le tableau de rotation des reviewers et l'ordre de choix des features sont construits pour l'attribution suivante :

1. **E1** : l'élève le plus expérimenté ;
2. **E2** : l'élève qui découvre Git et le développement ;
3. **E3, E4, E5** : les trois autres élèves.

Avec cette attribution :

1. E2 choisit sa première feature en premier, et dispose donc de toutes les features de niveau 1 ;
2. en partie 2, E2 relit la pull request de E4 avec E1, puis celle de E5 avec E3 : il n'est jamais seul reviewer d'une feature de code avant la partie 3 ;
3. en partie 3, la seconde feature de E2 est relue par E1 ;
4. E1 choisit sa seconde feature en premier au point d'étape 2, et peut prendre une feature de niveau 3 dans une zone déjà modifiée, ce qui garantit au moins un conflit à résoudre.

9. Contenu de l'archive

L'archive `memo-git.zip` contient le projet complet, sans le dossier `node_modules` ni historique Git :

<code>src/</code>	le site (HTML, CSS, JavaScript, pages Markdown)
<code>tests/</code>	30 tests unitaires (node:test)
<code>scripts/serve.mjs</code>	serveur de développement sans dépendance
<code>.github/</code>	modèle de description des pull requests
<code>.sonarcloud.properties</code>	configuration de l'analyse automatique
<code>README.md</code> , <code>CONTRIBUTING.md</code> , <code>CONTRIBUTEURS.md</code>	
<code>eslint.config.js</code> , <code>.prettierrc.json</code> , <code>.editorconfig</code> , <code>.gitattributes</code> , <code>.gitignore</code>	
<code>package.json</code> , <code>package-lock.json</code>	